

Sumario

(Dica: Clique com botão direito no sumario e selecione 'Atualizar Campo' para atualizar os números de pagina)

Sumario.....	2
1. Introducao	4
1.1 Objetivos.....	4
2. Arquitetura do Apache Flink	4
2.1 Componentes Principais	5
JobManager	5
TaskManager.....	5
Client	5
2.2 Modelo de Processamento.....	6
3. Instalacao e Configuracao.....	6
3.1 Requisitos do Sistema.....	7
3.2 Instalacao Passo a Passo.....	7
Etapa 1: Download.....	7
Etapa 2: Extracao	7
Etapa 3: Configuracao	7
Etapa 4: Iniciar Cluster	8
Etapa 5: Verificacao	8
3.3 Configuracao do Cluster	8
4. Camadas de Processamento	9
4.1 Camada ETL	10
Exemplo de Pipeline ETL	10
4.2 Camada de Machine Learning	11
Algoritmos Suportados	11
4.3 Camada de Analise de Dados	11
Exemplo Flink SQL.....	11
5. Tecnologias Emergentes	13
5.1 Apache Paimon.....	13

5.2 Stateful Functions.....	13
5.3 Integracao com LLMs.....	14
5.4 Kubernetes Native	14
6. Conclusao	15
Principais Beneficios.....	15
Proximos Passos Recomendados	15
Referencias e Bibliotecas.....	16
Documentacao Oficial	16
Livros Recomendados.....	16
Artigos e Publicacoes.....	16
Comunidade e Suporte.....	16
Dependencias Maven	16

1. Introdução

O Apache Flink é uma plataforma de processamento de dados unificada de código aberto, projetada para processar fluxos de dados em tempo real e em lote com alta performance. Desenvolvido originalmente pela equipe de pesquisa da TU Berlin em 2009 e posteriormente doado a Apache Software Foundation em 2014, o Flink tornou-se um dos frameworks mais populares para processamento de dados em grande escala.

Esta Prova de Conceito (POC) tem como objetivo demonstrar a instalação, configuração e operação da plataforma Apache Flink, abordando desde os conceitos fundamentais até implementações práticas em camadas de ETL, Machine Learning e Análise de Dados.

1.1 Objetivos do documento de projeto

1. Apresentar a arquitetura e componentes do Apache Flink
2. Demonstrar o processo de instalação e configuração
3. Explorar as camadas de processamento de dados
4. Identificar tecnologias emergentes para integração

1.1 Objetivos do projeto entregáveis

1. Introdução sobre a solução e Benefícios
2. Objetivo do Projeto
3. Descrição da plataforma
4. BPV (Business Process View) automação de negócios e diagrama
5. Fluxograma lógico de Instalação e Configuração
6. Referências e Pesquisa

2. Arquitetura do Apache Flink

A arquitetura do Apache Flink é projetada para fornecer processamento de dados distribuído, tolerante a falhas e com alto throughput. A plataforma suporta tanto processamento de streams quanto processamento batch através de uma API unificada.

A Figura 1 apresenta a arquitetura geral do Apache Flink:

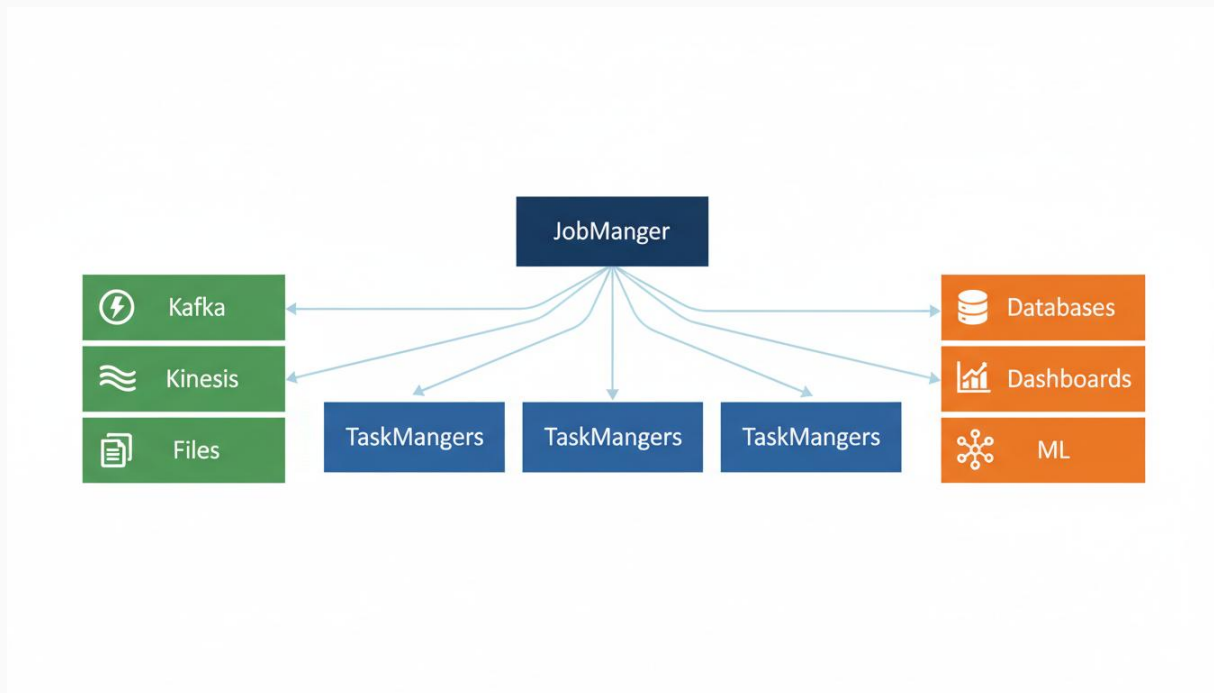


Figura 1: Arquitetura do Apache Flink

2.1 Componentes Principais

JobManager

O JobManager é o componente central que coordena a execução de jobs no cluster Flink. Suas responsabilidades incluem: agendamento de tarefas, coordenação de checkpoints, gerenciamento de recursos e recuperação de falhas. O JobManager mantém o estado global da aplicação e toma decisões sobre a distribuição de tarefas entre os TaskManagers.

TaskManager

Os TaskManagers são os nós de trabalho que executam as tarefas específicas de processamento de dados. Cada TaskManager possui um número configurável de slots de tarefas, onde cada slot pode executar uma cadeia de operadores. Os TaskManagers comunicam-se entre si para trocar dados durante o processamento distribuído.

Client

O Client é responsável por enviar jobs para o cluster Flink. Ele prepara o ambiente de execução, configura os parâmetros do job e submete o código para o JobManager. O client pode ser executado de forma independente do cluster, permitindo flexibilidade no desenvolvimento e teste.

2.2 Modelo de Processamento

O Flink utiliza um modelo de processamento de fluxo verdadeiro (true streaming), onde cada evento é processado individualmente assim que chega ao sistema. Isso contrasta com o modelo de micro-batches utilizado por outras plataformas como Apache Spark Streaming.

A Figura 2 ilustra o fluxo de processamento de dados no Flink:

Fluxograma técnico de processamento de dados em Apache Flink



Figura 2: Fluxo de Processamento de Dados

3. Instalação e Configuração

A instalação do Apache Flink pode ser realizada de diversas formas, desde configurações locais para desenvolvimento até clusters distribuídos em produção. Esta seção apresenta o processo completo de instalação e configuração.

A Figura 3 apresenta o fluxo de instalação e configuração:

Fluxograma de instalação e configuração Apache Flink em 6 etapas

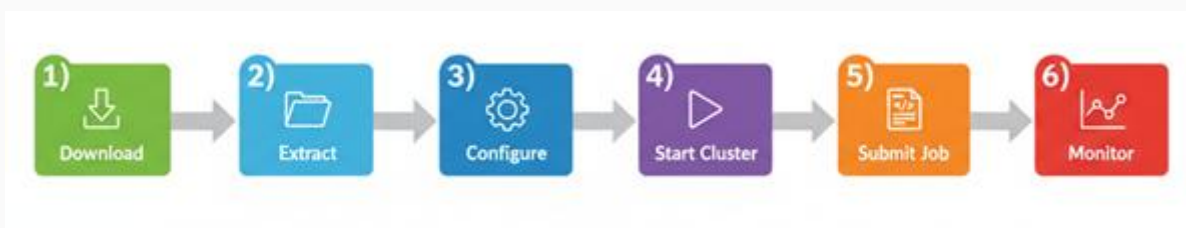


Figura 3: Fluxo de Instalação e Configuração

3.1 Requisitos do Sistema

Os requisitos mínimos para instalação do Apache Flink são:

Componente	Requisito
Java	JDK 11 ou superior
Memória RAM	Mínimo 4GB (recomendado 8GB+)
Disco	10GB de espaço livre
Sistema Operacional	Linux, macOS, Windows
Rede	Conectividade entre nós do cluster

Tabela 1: Requisitos do Sistema

3.2 Instalação Passo a Passo

Etapa 1: Download

Faça o download da versão mais recente do Apache Flink no site oficial. Recomenda-se a versão 2.0 ou superior para acesso aos recursos mais recentes:

```
wget https://dlcdn.apache.org/flink/flink-2.0.0/flink-2.0.0-bin-scala_2.12.tgz
```

Etapa 2: Extração

Extraia o arquivo baixado para o diretório desejado:

```
tar -xzf flink-2.0.0-bin-scala_2.12.tgz cd flink-2.0.0
```

Etapa 3: Configuração

Edite o arquivo flink-conf.yaml para configurar o cluster:

```
# configuracoes_basicas:  
jobmanager:  
  rpc:  
    address: localhost  
    port: 6123
```

```

memory:
  process:
    size: 1600m
taskmanager:
  memory:
    process:
      size: 1728m
  numberOfTaskSlots: 4
parallelism:
  default: 2

```

Etapa 4: Iniciar Cluster

Inicie o cluster Flink local:

```
./bin/start-cluster.sh
```

Etapa 5: Verificação

Acesse a interface web do Flink em <http://localhost:8081> para verificar se o cluster está funcionando corretamente.

3.3 Configuração do Cluster

Para ambientes de produção, é recomendado configurar um cluster distribuído com múltiplos TaskManagers. A configuração pode ser realizada através de arquivos de configuração ou utilizando orquestradores como Kubernetes.

A Figura 4 ilustra a arquitetura de deployment em Kubernetes:

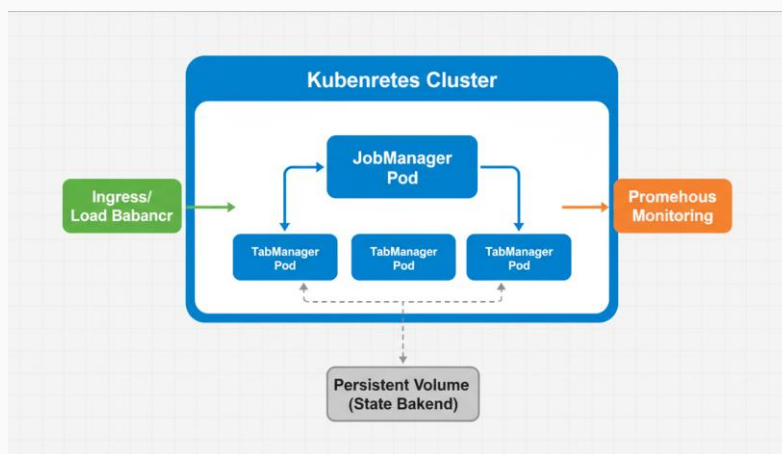


Figura 4: Deployment em Kubernetes

4. Camadas de Processamento

O Apache Flink suporta diversas camadas de processamento, permitindo a implementação de pipelines completos de dados desde a ingestão até a análise e machine learning.

A Figura 5 apresenta a arquitetura em camadas do Apache Flink:

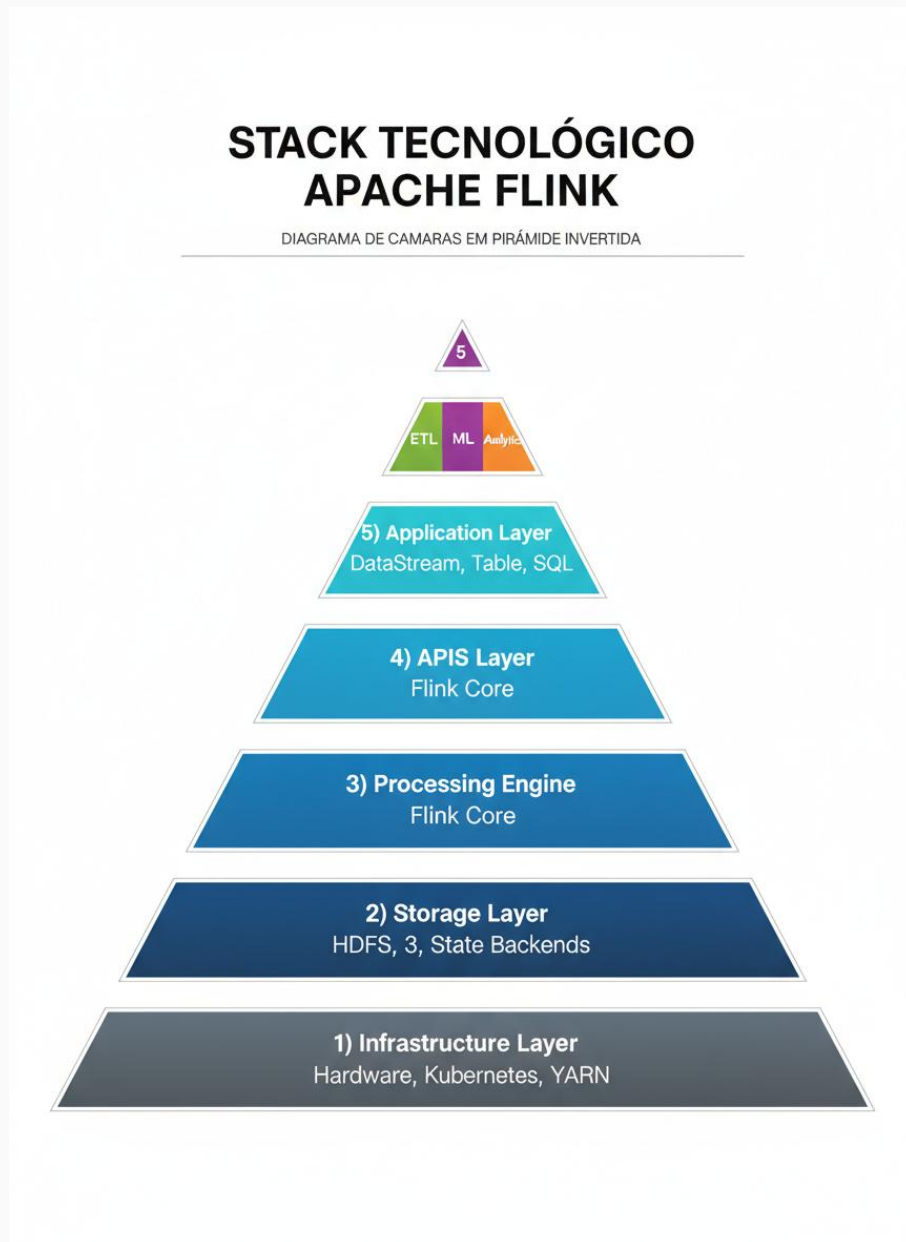


Figura 5: Arquitetura em Camadas

4.1 Camada ETL

A camada ETL (Extract, Transform, Load) é responsável pela ingestão, transformação e carga de dados. O Flink oferece conectores nativos para diversas fontes de dados, incluindo Apache Kafka, Amazon Kinesis, JDBC (Java Database Connectivity), Elasticsearch e sistemas de arquivos.

Exemplo de Pipeline ETL

```
// 1. Ambiente de execução
StreamExecutionEnvironment env = StreamExecutionEnvironment.getExecutionEnvironment();

// 2. Fonte de dados (extração do Kafka)
FlinkKafkaConsumer<String> kafkaSource = new FlinkKafkaConsumer<>(
    "input-topic",
    new SimpleStringSchema(),
    kafkaProperties
);

// 3. Pipeline de transformação
DataStream<Event> events = env
    .addSource(kafkaSource)           // Consome mensagens do Kafka
    .map(new JsonParser())           // Converte JSON em objeto Event
    .filter(event -> event.getValue() > 0) // Filtra eventos válidos
    .keyBy(Event::getCategory)       // Agrupa por categoria
    .window(TumblingEventTimeWindows.of(Time.minutes(5))) // Janela de 5 minutos
    .aggregate(new CountAggregate()); // Agrega contagem por janela
```

Estrutura lógica

- **Ambiente:** inicializa o `StreamExecutionEnvironment`.
- **Extração:** define a fonte de dados (Kafka).
- **Transformação:** aplica operações de parsing, filtragem, agrupamento e agregação.
- **Janela:** usa `TumblingEventTimeWindows` para processar em blocos fixos de tempo.
- **Carga:** resultado é um `DataStream` com os eventos agregados.

Esse formato deixa claro cada etapa do ETL: **extração → transformação → carga**.

4.2 Camada de Machine Learning

O Flink-ML é a biblioteca de machine learning do Apache Flink, projetada para fornecer algoritmos de ML escaláveis com suporte a processamento em tempo real. A biblioteca oferece APIs tanto para Java quanto para Python.

A Figura 6 ilustra o pipeline de Machine Learning com Flink-ML:

Fluxograma de processamento de Machine Learning com Flink ML

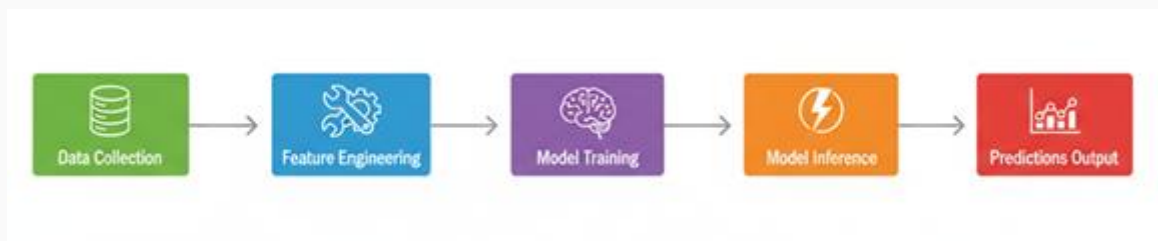


Figura 6: Pipeline de Machine Learning

Algoritmos Suportados

5. Regressão Linear e Logística
6. K-Means Clustering
7. Support Vector Machines (SVM)
8. Algoritmos de Recomendação
9. Feature Engineering

4.3 Camada de Análise de Dados

A camada de análise de dados utiliza Flink SQL e Table API para consultas analíticas sobre streams e batches de dados. O Flink SQL é compatível com o padrão ANSI SQL, facilitando a migração de análises existentes.

Exemplo Flink SQL

```
-- 1. Criação da tabela de eventos (Fonte: Kafka)
CREATE TABLE events (
  user_id STRING,
  event_type STRING,
```

```

    event_time TIMESTAMP(3),
    value DECIMAL(10,2),
    WATERMARK FOR event_time AS event_time - INTERVAL '5' SECOND
) WITH (
    'connector' = 'kafka',
    'topic' = 'user-events',
    'properties.bootstrap.servers' = 'localhost:9092',
    'format' = 'json'
);
-- 2. Consulta analítica com janela de 1 hora
SELECT
    TUMBLE_START(event_time, INTERVAL '1' HOUR) AS window_start,
    event_type,
    COUNT(*) AS event_count,
    AVG(value) AS avg_value
FROM events
GROUP BY
    TUMBLE(event_time, INTERVAL '1' HOUR),
    event_type;

```

Estrutura lógica

- **Fonte de dados (Kafka):** a tabela events é criada com schema definido e watermark para controle de tempo de evento.
- **Janela temporal:** TUMBLE define janelas fixas de 1 hora.
- **Agregações:**
 - COUNT(*) → número de eventos por tipo.
 - AVG(value) → média dos valores por tipo.
- **Resultado:** relatório por tipo de evento, agrupado em janelas horárias.

Esse formato deixa claro o fluxo: **ingestão → definição de tempo → agregação → resultado analítico.**

5. Tecnologias Emergentes

O ecossistema do Apache Flink continua evoluindo com novas tecnologias e integrações. Esta seção apresenta as tecnologias emergentes mais relevantes para o ambiente Flink.

A Figura 7 apresenta o ecossistema de integrações do Apache Flink:



Figura 7: Ecossistema Apache Flink

5.1 Apache Paimon

O Apache Paimon (anteriormente Flink Table Store) é um formato de tabela de streaming que combina os benefícios de data lakes e data warehouses. Ele oferece armazenamento de dados em tempo real com alta performance de consulta.

5.2 Stateful Functions

As Stateful Functions permitem construir aplicativos distribuídos stateful com abstrações de alto nível. Esta API simplifica o desenvolvimento de aplicativos complexos como micro serviços orquestrados e máquinas de estado.

5.3 Integração com LLMs

A partir da versão 2.1, o Flink suporta integração com Large Language Models (LLMs) através da função `ML_PREDICT` no Flink SQL. Isso permite realizar análise semântica e processamento de linguagem natural em pipelines de dados.

5.4 Kubernetes Native

O Flink Kubernetes Operator fornece gerenciamento nativo de clusters Flink no Kubernetes, permitindo deployment, escalonamento automático e gerenciamento de ciclo de vida de aplicações Flink de forma declarativa.

Tecnologia	Descrição	Versão
Apache Paimon	Formato de tabela streaming	0.8+
Flink Operator	Gerenciamento K8s nativo	1.8+
Flink-ML	Biblioteca de ML	2.3+
Stateful Functions	Aplicativos stateful	3.3+

Tabela 2: Tecnologias Emergentes

6. Conclusão

Esta Prova de Conceito demonstrou a instalação, configuração e operação da plataforma Apache Flink, apresentando sua arquitetura distribuída, modelos de processamento e camadas de aplicação para ETL, Machine Learning e Análise de Dados.

O Apache Flink destaca-se como uma plataforma unificada de processamento de dados, oferecendo verdadeiro processamento de streams com baixa latência, garantias exatamente-uma-vez e integração com diversas tecnologias do ecossistema de big data.

Principais Benefícios

10. Processamento unificado de batch e streaming
11. Baixa latência e alto throughput
12. Tolerância a falhas com checkpoints
13. APIs em Java, Python e SQL
14. Ecossistema rico de conectores

Próximos Passos Recomendados

Para implementação em produção, recomenda-se:

- (1) Configurar High Availability para o JobManager,
- (2) Implementar monitoramento com Prometheus e Grafana,
- (3) Configurar State Backends apropriados,
- (4) Realizar testes de carga
- (5) Estabelecer procedimentos de backup e recuperação.

Referencias e Bibliotecas

Documentação Oficial

Apache Flink Documentation:<https://nightlies.apache.org/flink/flink-docs-stable/>

Flink ML Documentation:<https://nightlies.apache.org/flink/flink-ml-docs-stable/>

Flink Kubernetes Operator:<https://nightlies.apache.org/flink/flink-kubernetes-operator-docs-stable/>

Livros Recomendados

Stream Processing with Apache Flink:Fabian Hueske & Vasiliki Kalavri, O'Reilly Media

Learning Apache Flink:Tanmay Deshpande, Packt Publishing

Artigos e Publicações

Carbone et al. (2017): State management in Apache Flink. Proceedings of the VLDB Endowment.

Flink-ML (2024): Machine Learning in Apache Flink. Brazilian Journal of Technology.

Comunidade e Suporte

Apache Flink Mailing Lists:<https://flink.apache.org/community.html>

GitHub - Apache Flink:<https://github.com/apache/flink>

GitHub - Flink-ML:<https://github.com/apache/flink-ml>

Dependências Maven

```
<dependencies>
  <!-- Flink Streaming API -->
  <dependency>
    <groupId>org.apache.flink</groupId>
    <artifactId>flink-streaming-java</artifactId>
    <version>2.0.0</version>
  </dependency>
  <!-- Conector Kafka para Flink -->
  <dependency>
    <groupId>org.apache.flink</groupId>
```



```
<artifactId>flink-connector-kafka</artifactId>
<version>3.2.0</version>
</dependency>
<!-- Flink Machine Learning Core -->
<dependency>
  <groupId>org.apache.flink</groupId>
  <artifactId>flink-ml-core</artifactId>
  <version>2.3.0</version>
</dependency>
</dependencies>
```

Estrutura lógica

- **flink-streaming-java** → fornece a API de streaming para construir pipelines de dados.
- **flink-connector-kafka** → habilita integração com tópicos Kafka como fonte ou destino.
- **flink-ml-core** → adiciona funcionalidades de machine learning dentro do Flink.

Assim você tem um bloco único que pode ser colocado diretamente no pom.xml, mantendo tudo organizado e fácil de manter.

